UNIVERSIDAD DE MURCIA

GRADO EN INGENIERÍA INFORMÁTICA

SERVICIOS TELEMÁTICOS
PRÁCTICA FINAL 25/26

Gonzalo Lucena Vázquez
23833365Y
gonzalo.lucenav@um.es

Grupo de prácticas 3.1

Eduardo Terry Gavilá
24420735W
eduardo.terryg@um.es

Grupo de prácticas 3.4

Profesor responsable: Fernando Pereñíguez García

Fecha: 23/04/2026

1. Introducción **2**
2. Servidor Web HTTP **3**
3. DNS y SMTP/POP **10**
4. Servicio HTTPS **14**
5. IPSec **17**
6. Funcionalidad opcional añadida **20**
6.1. Instalar y configurar IMAP como servidor de correo entrante 20
6.2. Desplegar un dominio de resolución inversa 21
6.3. Crear un subdominio para nombreorganizacion6535.org e instalar un DNS para el mismo 23
6.4. Realizar la autenticación del cliente con certificados en HTTPS 25
6.5. Instalar y configurar FTP 29
7. Trazas de Wireshark **31**
8. Problemas encontrados **37**
9. Conclusión y valoración personal **38**
10. Bibliografía **39**

1. Introducción

En el siguiente documento vamos a explicar cómo hemos llevado a cabo la práctica de la asignatura Servicios Telemáticos 2025/26.
En cada uno de los siguientes apartados se explicarán los cambios más relevantes que hemos implementado en cada uno de los ficheros de configuración mencionados tanto en los laboratorios de prácticas como en las diapositivas disponibles en el apartado de recursos de la asignatura.

La memoria se divide en:

- Servidor Web HTTP: Explicación de cómo hemos desarrollado el servidor web Python.
- DNS y SMTP/POP: Se detalla la configuración del DNS con arquitectura de servidor primario y secundario. Además, se comenta la integración de los servicios de correo electrónico usando SMTP/POP.
- Servicio HTTPS: Proceso de la configuración del servidor Apache para obtener servicios web cifrados utilizando certificados.
- IPSec: Descripción de la implementación del estándar IPSec para establecer una comunicación segura de extremo a extremo entre las máquinas virtuales, utilizando autenticación mediante certificados.
- Funcionalidad opcional añadida: Apartados opcionales añadidos entre los que se encuentran:
    - Instalar y configurar IMAP como servidor de correo entrante.
    - Desplegar un dominio de resolución inversa.
    - Crear un subdominio para nombreorganizacion6535.org e instalar un DNS para el mismo
    - Realizar la autenticación de cliente con certificados en HTTPS.
    - Instalar y configurar FTP.
- Trazas de Wireshark: Incluye las trazas de Wireshark más relevantes obtenidas durante la realización del proyecto.

Las versiones y software utilizados son en todo caso los indicados por el profesor de prácticas y diapositivas.

2. Servidor Web HTTP

En primer lugar se va a describir cómo se ha llevado a cabo el desarrollo mediante Python para el funcionamiento de nuestro servidor web.

Inicialmente, se pueden ver tanto los imports, siendo críticos tanto el import de sockets para los sockets TCP como el import del select para implementar la persistencia. El resto de imports son más específicos para mostrar cierta información, formatos o para analizar las expresiones regulares, cada uno tiene un comentario especificándolo.

Debajo se encuentran la definición de constantes, que se nos especifican a través del enunciado de la práctica, el conjunto de extensiones admitidas por HTTP (de texto y de imágenes) y la configuración del logger del servidor con lo que irá mostrando cada vez que haya una petición:

```python
# coding=utf-8
#!/usr/bin/env python3

import socket
import selectors    #https://docs.python.org/3/library/selectors.html
import select
import types        # Para definir el tipo de datos data
import argparse     # Leer parametros de ejecución
import os           # Obtener ruta y extension
from datetime import datetime, timedelta, timezone # Fechas de los mensajes HTTP
import time         # Timeout conexión
import sys          # sys.exit
import re           # Analizador sintáctico
import logging      # Para imprimir logs



BUFSIZE = 8192 # Tamaño máximo del buffer que se puede utilizar
TIMEOUT_CONNECTION = 29 # Timout para la conexión persistente - 6 + 5 + 3 + 5 + 10
MAX_ACCESOS = 10

# Extensiones admitidas (extension, name in HTTP)
filetypes = {"gif":"image/gif", "jpg":"image/jpg", "jpeg":"image/jpeg", "png":"image/png", "htm":"text/htm",
             "html":"text/html", "css":"text/css", "js":"text/js"}

# Configuración de logging
logging.basicConfig(level=logging.INFO,
                    format='[%(asctime)s.%(msecs)03d] [%(levelname)-7s] %(message)s',
                    datefmt='%Y-%m-%d %H:%M:%S')
logger = logging.getLogger()
```

imagen 1: constantes del servidor Python.

Tras eso, están las declaraciones de funciones en las que se apoyará el main. Analizándolas una a una tenemos:
→ **enviar_mensaje** se encarga de enviar el string de datos codificado como una secuencia de bytes.

```python
def enviar_mensaje(cs, data):
    # Convertimos a bytes si es necesario
    if type(data) == str:
        data = data.encode()
    return cs.send(data)
```

imagen 2: función enviar_mensaje.

→ **recibir_mensaje** realiza la lectura del mensaje de 8KB en 8KB (Bufsize) de datos para posteriormente decodificarlo.

```python
def recibir_mensaje(cs):
    # Leemos hasta BUFSIZE bytes
    data = cs.recv(BUFSIZE)
    return data.decode()    # Devolvemos el string decodificado
```

imagen 3: función recibir_mensaje.

→ **cerrar_conexion**, que simplemente cierra la conexión TCP establecida.

```python
def cerrar_conexion(cs):
    cs.close()
```

imagen 4: función cerrar_conexion.

Y por último las 2 funciones claves de nuestro servidor:

→ **process_cookies.** Esta función es la encargada de ir leyendo y actualizar la cookie para el control de acceso.

Primero recorre las cabeceras en busca de la que comience por "Cookie". Una vez la encuentra, divide la cadena mediante el método split, aislando los diferentes pares clave-valor hasta encontrar nuestra cookie_counter_6535.

Tras ello, convierte el valor de la cookie a entero y evalúa:

- Si el valor es menor que MAX_ACCESOS (10), lo incrementa y lo devuelve.
- Si el valor es mayor o igual que MAX_ACCESOS, devuelve MAX_ACCESOS y eso hace que process_web_request intercepte la petición antes de siquiera servir el fichero y devuelva el código 403 Forbidden. Cerrando así la conexión y denegando el acceso hasta que la cookie se restaure al valor inicial.

Y si no encuentra la cookie en las cabeceras, devuelve 1.

```python
def process_cookies(headers, is_index):
    nombre_cookie = "cookie_counter_6535"

    # Se analizan las cabeceras en headers para buscar la cabecera cookie
    for linea in headers:
        if linea.startswith("Cookie:"):
            # Obtenemos el contenido de la cabecera
            cookie_header_content = linea.split(":", 1)[1].strip()

            # Separamos las cookies individuales
            cookies = cookie_header_content.split(";")

            for cookie in cookies:
                if "=" in cookie:
                    key, value = cookie.split("=", 1)
                    key = key.strip()
                    value = value.strip()

                    # Comprobamos si es la cookie que buscamos
                    if key == nombre_cookie:
                        try:
                            val = int(value)

                            # Si se encuentra y tiene el valor MAX_ACCESOS se devuelve MAX_ACCESOS
                            if val >= MAX_ACCESOS:
                                return MAX_ACCESOS

                            # Si se encuentra, tiene un valor 1 <= x < MAX_ACCESOS
                            # y la petición es al index, se incrementa en 1
                            if is_index and 1 <= val < MAX_ACCESOS:
                                val += 1

                            return val
                        except ValueError:
                            return 1

    return 1
```

imagen 5: función process_cookies.

→ process_web_request

Esta es nuestra función más importante, ya que es donde tenemos las partes más críticas de la funcionalidad. Se encarga de interpretar el protocolo HTTP, servir archivos y gestionar la persistencia de la conexión TCP.

Vamos a analizarla por partes:

- Persistencia mediante keep-alive: Lo implementamos usando un bucle while controlado por un booleano "comprobación". Tras atender una petición, en vez de cerrar el socket, nuestro proceso llama al método select(), configurado con nuestro timeout personalizado (en nuestro escenario es 29s). Si el cliente envía una nueva petición a través del socket antes de que salte el timeout, repetimos el bucle. Y si salta el timeout sin haber recibido datos, se pone comprobación a true y se sale del bucle, cerrando así la conexión establecida.

```python
        # Persistencia
        if not comprobacion:
            recibido, _, _ = select.select([cs], [], [], TIMEOUT_CONNECTION)

            if not recibido:
                # Si recibido está vacío, significa que ha saltado el TIMEOUT
                logger.info("Timeout de persistencia alcanzado. Cerrando.")
                comprobacion = True

            # Si recibido tiene datos, el bucle while se repite
            # y recibir_mensaje leerá la nueva peticion inmediatamente
```

imagen 6: persistencia.

- Tratamiento de sintaxis y errores: Analizamos la primera línea de la petición HTTP, separando los campos Método, URL y Versión. Esto lo hacemos mediante expresiones regulares y el método fullmatch(), para asegurarnos que el formato es correcto. Se pueden dar varios casos:
    1) Si la versión es correcta (la 1.1) pero no se usa GET, se devuelve el error 405: Method not allowed
    2) Si se trata de un fallo de sintaxis, se devuelve el error 400: Bad request.
    3) Si el recurso que solicita el cliente no existe (nuestro servidor no tiene acceso a él), devuelve el error 404: Not found.

```python
lineas = data.split('\r\n', 1)
request_line = lineas[0]

partes = request_line.split(' ')
method = ""
url= ""
version= ""
request_valida = False

if len(partes) == 3:
    method = partes[0]  # GET
    url = partes[1]  # index.html
    version = partes[2] # 1.1

    er_method = r"^GET$"
    er_url = r"^\S+$"
    er_version = r"^HTTP/1\.1$"

    if (re.fullmatch(er_method, method) and
        re.fullmatch(er_url, url) and
        re.fullmatch(er_version, version)):
        request_valida = True

        tiene_host = False
        headers_block = lineas[1].split('\r\n') if len(lineas) > 1 else []
        for h in headers_block:
            if h.lower().startswith("host:"):
                tiene_host = True
                break
        if tiene_host:
            request_valida = True
        else:
            logger.error("Error: falta cabecera Host")

else:
    print("Error 400 (Bad request): La línea de petición no tiene el formato correcto")
```

```python
# Se comprueba si la petición es valida
if request_valida:
    mensaje_email = b""
    # Leer URL y eliminar parámetros si los hubiera
    if "?" in url:
        ruta_base, params = url.split("?",1)
        url = ruta_base # Nos quedamos con la ruta limpia para buscar el fichero
        params = params.replace("%40", "@")
        regex_correo = r"email=[a-z]([a-z.]*[a-z])?@nombreorganizacion6535\.org"
        if re.search(regex_correo, params):
            mensaje_email = b"<h1>Correo Correcto</h1><p>Bienvenido estudiante.</p>"
        elif "email=" in params: # Si mandó el parámetro email pero no cumple el patrón:
            mensaje_email = b"<h1>Correo Erroneo</h1><p>El correo debe ser valido, en minusculas y pertenecer al dominio @nombreorganizacion6535.org.</p>"

    # Comprobar si el recurso solicitado es /, en ese caso el recurso es index.html
    if url == "/":
        url = "/index.html"

    # Construir la ruta absoluta del recurso (webroot + recurso solicitado)
    filepath = os.path.join(webroot, url.lstrip("/"))

    # Comprobar que el fichero existe, si no devolver Error 404 "Not found"
    if not os.path.isfile(filepath):
        # ERROR 404
        logger.warning("Fichero no encontrado: {}".format(filepath))
        error_msg = "<h1>404 Not Found</h1><p>El recurso no existe.</p>"

        # Construccion cabecera:
        # Linea de estado:
        header = "HTTP/1.1 404 Not Found\r\n"
        # Fecha:
        header += "Date: {}\r\n".format(datetime.now(timezone.utc).strftime('%a, %d %b %Y %H:%M:%S GMT'))
        # Nombre del servidor:
        header += "Server: web.organizacion6535.org\r\n"
        # Content-Length:
        header += "Content-Length: {}\r\n".format(len(error_msg))
        # Conexion:
        header += "Connection: keep-alive\r\n"
        # Content-type:
        header += "Content-Type: text/html\r\n"

        # Linea vacia:
        header += "\r\n"

        enviar_mensaje(cs, header.encode() + error_msg.encode())
```

```python
else:
    if len(partes) == 3 and re.fullmatch(er_version, version) and not re.fullmatch(er_method, method):
        error_msg = "405 Method Not Allowed. Metodo no permitido"
        header = "HTTP/1.1 405 Method Not Allowed\r\n"
        header += "Allow: GET\r\n"
        header += "Content-Length: {}\r\n".format(len(error_msg))
        header += "Connection: close\r\n\r\n"
        print("Error 405 (Method Not Allowed): Método no permitido, solo se acepta GET")
        enviar_mensaje(cs, header.encode() + error_msg.encode())
    else:
        # 400 Bad Request
        error_msg = "<h1>400 Bad Request</h1>"
        header = "HTTP/1.1 400 Bad Request\r\n"
        header += "Content-Length: {}\r\n".format(len(error_msg))
        header += "Connection: close\r\n\r\n"
        enviar_mensaje(cs, header.encode() + error_msg.encode())

    comprobacion = True
```

imágenes 7, 8 y 9: tratamiento de sintaxis y errores.

- Gestión de los formularios GET: Si la URL contiene algún parámetro, lo cual viene indicado por el carácter "?", separamos la ruta base de los datos. Decodificamos caracteres especiales de la URL, como pasar el @ a %40 y, además, empleamos una expresión regular para validar que los correos introducidos sean válidos bajo el estándar de la UMU y, mostrando un mensaje HTML en respuesta.

```
# Se comprueba si la petición es valida
if request_valida:
    mensaje_email = b""
    # Leer URL y eliminar parámetros si los hubiera
    if "?" in url:
        ruta_base, params = url.split("?",1)
        url = ruta_base # Nos quedamos con la ruta limpia para buscar el fichero
        params = params.replace("%40", "@")
        regex_correo = r"email=[a-z]([a-z.]*[a-z])?@nombreorganizacion6535\.org"
        if re.search(regex_correo, params):
            mensaje_email = b"<h1>Correo Correcto</h1><p>Bienvenido estudiante.</p>"
        elif "email=" in params: # Si mandó el parámetro email pero no cumple el patrón:
            mensaje_email = b"<h1>Correo Erroneo</h1><p>El correo debe ser valido, en minusculas y pertenecer al dominio @nombreorganizacion6535.org.</p>"
```

imagen 10: gestión de los GET

- Generación de HTTP Responses (código 200 OK): Si la petición HTTP es válida, se procede a construir las cabeceras de la respuesta HTTP de nuestro servidor: Server, Date, Content-Length, Content-Type, Connection y Keep-Alive. Tras eso, el contenido solicitado se va enviando en bloques de tamaño de 8KB (BUFSIZE).

```
        else:
            # ÉXITO (200 OK)
            file_size = os.path.getsize(filepath)
            filename, file_extension = os.path.splitext(filepath)
            extension = file_extension.lstrip(".")
            content_type = filetypes.get(extension, "application/octet-stream")

            # Construimos cabeceras 200 OK
            header = "HTTP/1.1 200 OK\r\n"
            header += "Date: {}\r\n".format(datetime.now(timezone.utc).strftime('%a, %d %b %Y %H:%M:%S GMT'))
            header += "Server: web.organizacion6535.org\r\n"
            header += "Content-Length: {}\r\n".format(file_size + len(mensaje_email))
            header += "Content-Type: {}\r\n".format(content_type)

            if url == "/index.html":
                header += "Set-Cookie: cookie_counter_6535={}; Max-Age=30\r\n".format(cookie_val)

            header += "Connection: keep-alive\r\n"
            header += "Keep-Alive: timeout={}, max=100\r\n\r\n".format(TIMEOUT_CONNECTION)

            # Enviamos cabeceras y elaviso del correo
            enviar_mensaje(cs, header.encode() + mensaje_email)

                # Enviar contenido del fichero por bloques
            with open(filepath, 'rb') as f:
                while True:
                    chunk = f.read(BUFSIZE)
                    if not chunk:
                        break
                    cs.send(chunk)
```

imagen 11: generación de Responses (200 OK)

→ **método main,** actúa como el gestor de conexiones del servidor.
Abre un socket TCP y se le aplica la opción SO_REUSEADDR para evitar problemas de puerto bloqueado al reiniciar el servicio rápidamente. Además, se le hace un bind a la IP y puertos especificados como argumentos y se pone a escuchar conexiones con listen().

También, para aceptar varios clientes simultáneamente, se hace un bucle infinito que va realizando accepts. Por cada nueva conexión entrante, se genera un proceso hijo mediante

os.fork() y el hijo toma el control de la conexión que ha entrado llamando a process_web_request(), mientras que el padre cierra su conexión con ese cliente y vuelve inmediatamente a escuchar nuevas peticiones.

3. DNS y SMTP/POP

A continuación se describen las configuraciones adoptadas para la realización de consultas DNS y envíos de correo con SMTP y POP.

(Algunas diferencias en los nombres o direcciones IPs se debe a que unas capturas han sido tomadas por Eduardo y otras por Gonzalo).

Primeramente, el fichero /etc/hosts tanto en los servidores (el que actúa como DNS primario y el que lo hace como secundario) como en el cliente deben estar de fábrica:

```
  GNU nano 2.5.3            Archivo: /etc/hosts

127.0.0.1       localhost
127.0.1.1       desktop
#192.168.230.5  www.webprueba.com
#192.168.230.5  smtp.st.um st.um
#192.168.230.5  pop.st.um st.um
```

imagen 12: /etc/hosts en el cliente

```
  GNU nano 2.5.3                    Archivo: /etc/hosts

127.0.0.1       localhost
127.0.1.1       server1
```

```
  GNU nano 2.5.3                    Archivo: /etc/hosts

127.0.0.1       localhost
127.0.1.1       server2
```

imágenes 13 y 14: /etc/hosts en los dos servidores

También, en el cliente, hemos editado el fichero /etc/resolv.conf de una manera especial, ya que como este fichero se formatea cada vez que se inicia la máquina virtual, lo cambiamos a través del fichero /etc/resolvconf/resolv.conf/head. Esto lo que incluye son las identificaciones de las direcciones de nuestros dos servidores como servidores DNS:

```
alumno@desktop: ~
  GNU nano 2.5.3    Archivo: /etc/resolvconf/resolv.conf.d/head

# Dynamic resolv.conf(5) file for glibc resolver(3) generated by resolvconf(8)
#     DO NOT EDIT THIS FILE BY HAND -- YOUR CHANGES WILL BE OVERWRITTEN
nameserver 192.168.230.5
nameserver 192.168.230.6
```

imagen 15: archivo head de resolv.conf

Ya en el servidor DNS primario (análogo en el secundario), el archivo named.conf.options ha sido ligeramente editado, cambiando simplemente la opción de dnssec-validation a no:

```
  GNU nano 2.5.3              Archivo: /etc/bind/named.conf.options

options {
        directory "/var/cache/bind";

        // If there is a firewall between you and nameservers you want
        // to talk to, you may need to fix the firewall to allow multiple
        // ports to talk.  See http://www.kb.cert.org/vuls/id/800113

        // If your ISP provided one or more IP addresses for stable
        // nameservers, you probably want to use them as forwarders.
        // Uncomment the following block, and insert the addresses replacing
        // the all-0's placeholder.

        // forwarders {
        //      0.0.0.0;
        // };

        //========================================================================
        // If BIND logs error messages about the root key being expired,
        // you will need to update your keys.  See https://www.isc.org/bind-keys
        //========================================================================
        dnssec-validation no;

        auth-nxdomain no;    # conform to RFC1035
```

imagen 16: configuración de named.conf.options

Ahora, en cuanto al named.conf.local sí existen diferencias entre el servidor DNS primario y el secundario. El del primario luce así:

```
  GNU nano 2.5.3                Archivo: /etc/bind/named.conf.local

//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "nombreorganizacion6535.org." {
        type master;
        file "/etc/bind/db.nombreorganizacion6535.org.zone";
        allow-transfer { 192.168.230.6; };
};
```

imagen 17: configuración de named.conf.local en el servidor DNS primario

Aquí se especifica que es de tipo “master”, es decir, primario, se especifica donde se guarda el fichero de zona (en /etc/bind) y la sección de allow-transfer donde se incluye la ip del servidor DNS secundario para permitir las transferencias entre ellos.

En cambio el secundario tiene tipo “esclavo” que indica que es el secundario, el fichero de zona lo guarda en un lugar diferente (en /var/cache/bind) y en la sección de masters tiene la dirección IP de nuestro servidor DNS primario:

```
  GNU nano 2.5.3                Archivo: /etc/bind/named.conf.local

//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "nombreorganizacion6535.org." {
        type slave;
        file "/var/cache/bind/db.nombreorganizacion6535.org.zone";
        masters { 192.168.230.5; };
};
```

imagen 18: configuración de named.conf.local en el servidor DNS secundario

Este es el contenido de nuestro fichero de zona. Está ubicado en /etc/bind y su nombre es db.nombreorganizacion6535.org.
En este fichero se encuentran diferentes registros los cuales están configurados para el correcto funcionamiento del servidor.

El registro SOA se almacena en forma de bloque y de aquí lo único que hemos configurado a parte de poner servidor.nombreorganizacion6535.org. y admin.nombreorganizacion6536.org. ha sido poner la versión a 3 (primera fila). Esto sirve para comunicar al DNS secundario de las modificaciones de manera que pueda utilizar lo añadido.

Los registros NS (name server) indican cuales son los dominios que actúan como servidores. En nuestro caso, tenemos este mismo equipo (el de la captura) y el servidor clonado.

El registro MX el cual también incluye un número que indica la prioridad, sirve para indicar a donde deben de llegar los correos electrónicos los cuales están administrados con SMTP.

Los registros tipo A contienen la dirección la cual maneja los servicios a la izquierda de ellos.

```
;
; BIND data file for local loopback interface
;
$TTL    604800
$ORIGIN nombreorganizacion6535.org.

@       IN      SOA     servidor.nombreorganizacion6535.org. admin.nombreorganizacion6535.org. (
                              3         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      servidor.nombreorganizacion6535.org.
@       IN      NS      servidorclonado.nombreorganizacion6535.org.
@       IN      MX      10      smtp.nombreorganizacion6535.org.

servidor        IN      A       192.168.56.102
servidorclonado IN      A       192.168.56.103

cliente IN      A       192.168.56.102
dns1    IN      A       192.168.56.102
web     IN      A       192.168.56.102
www     IN      A       192.168.56.102
pop     IN      A       192.168.56.102
smtp    IN      A       192.168.56.102
dns2    IN      A       192.168.56.102
```

imagen 19: configuración del fichero de zona

4. Servicio HTTPS

En este apartado se describirán los cambios e implementaciones relacionadas con HTTPS. Para ello se ha usado apache como bien indica la práctica.

Lo primero una vez instalado es en la ruta /etc/apache2/sites-available crear el fichero .conf

El archivo .conf configura los VirtualHosts de Apache para servir la web tanto por HTTP como por HTTPS. En esta práctica, su función es configurar el cifrado en el puerto seguro indicando las rutas de las claves y establecer la directiva SSLVerifyClient require, lo que bloquea el acceso a cualquier usuario que no presente un certificado de cliente válido.

```
root@server:/etc/apache2/sites-available# cat nombreorganizacion6535.conf
<VirtualHost *:80>
        ServerAdmin admin@nombreorganizacion6535.org
        ServerName www.nombreorganizacion6535.org
        ServerAlias nombreorganizacion6535.org
        DocumentRoot /var/www/nombreorganizacion6535

        <Directory /var/www/nombreorganizacion6535>
                AllowOverride None
                Order allow,deny
                allow from all
        </Directory>
</VirtualHost>

<VirtualHost *:443>
        ServerAdmin edu@sstt.com
        ServerName www.nombreorganizacion6535.org
        ServerAlias nombreorganizacion6535.org
        DocumentRoot /var/www/nombreorganizacion6535

        <Directory /var/www/nombreorganizacion6535>
                AllowOverride None
                Order allow,deny
                allow from all
        </Directory>

        SSLEngine on
        SSLCertificateFile /home/alumno/demoCA/servercertSAN.pem
        SSLCertificateKeyFile /home/alumno/demoCA/serverkeySAN.pem
        SSLCACertificateFile /home/alumno/demoCA/cacert.pem
        SSLVerifyClient require
        SSLVerifyDepth 10
</VirtualHost>
```

imagen 20: fichero .conf de apache

Lo siguiente es analizar el directorio de trabajo de la CA:

```
root@server:/home/alumno/demoCA# ls
cacert.pem      clientcsr.pem  index.txt          newcerts    servercert.pem     serverkeySAN.pem
certs           clientkey.pem  index.txt.attr     private     servercertSAN.pem
clientcert.pem  confirg.cfg    index.txt.attr.old serial      servercsrSAN.pem
clientcert.pfx  crlnumber      index.txt.old      serial.old  serverkey.pem
```

imagen 21: directorio demoCA

cacert.pem: Es el certificado público raíz del CA. El navegador necesita este archivo para confiar en lo que se firme.

servercertSAN.pem: El certificado público del servidor web Apache.

serverkeySAN.pem: La clave privada del servidor web. Trabaja complementariamente con el certificado anterior para cifrar las conexiones y debe mantenerse oculta.

servercsrSAN.pem: La petición de firma (CSR) original que se generó para que la CA emitiera el certificado del servidor web.

private: Carpeta restringida donde se aloja la clave privada de la CA (cakey.pem), la cual se usa cada vez que se ejecuta un comando para firmar un nuevo certificado.

Otra modificación interesante es la del fichero /usr/lib/ssl/openssl.cnf.

```
[ req_distinguished_name ]
countryName                     = Country Name (2 letter code)
countryName_default             = ES
countryName_min                 = 2
countryName_max                 = 2

stateOrProvinceName             = State or Province Name (full name)
stateOrProvinceName_default     = Murcia

localityName                    = Locality Name (eg, city)

0.organizationName              = Organization Name (eg, company)
0.organizationName_default      = UMU

# we can do this but it is not needed normally :-)
#1.organizationName             = Second Organization Name (eg, company)
#1.organizationName_default     = World Wide Web Pty Ltd

organizationalUnitName          = Organizational Unit Name (eg, section)
organizationalUnitName_default  = nombreorganizacion6535

commonName                      = Common Name (e.g. server FQDN or YOUR name)
commonName_max                  = 64

emailAddress                    = Email Address
```

imagen 22: fichero openssl.cnf

Establecer estos valores por defecto en el fichero openssl.cnf agiliza la creación de peticiones de firma al completar automáticamente los datos habituales de la organización. Esto asegura la consistencia de la información en todos los certificados emitidos por la CA y reduce la posibilidad de cometer errores manuales durante la generación de estos.

Ya solo queda ejecutar todos los comandos mencionados en el guión e importar el certificado en el navegador.

Lo que hay que hacer para comprobar que todo funciona correctamente es introducir en el buscador lo siguiente: https://www.nombreorganizacion6535.org
Al hacerlo, nos llevará a una pequeña web de prueba en la que se nos indica que hemos conseguido acceder y el servidor funciona correctamente.

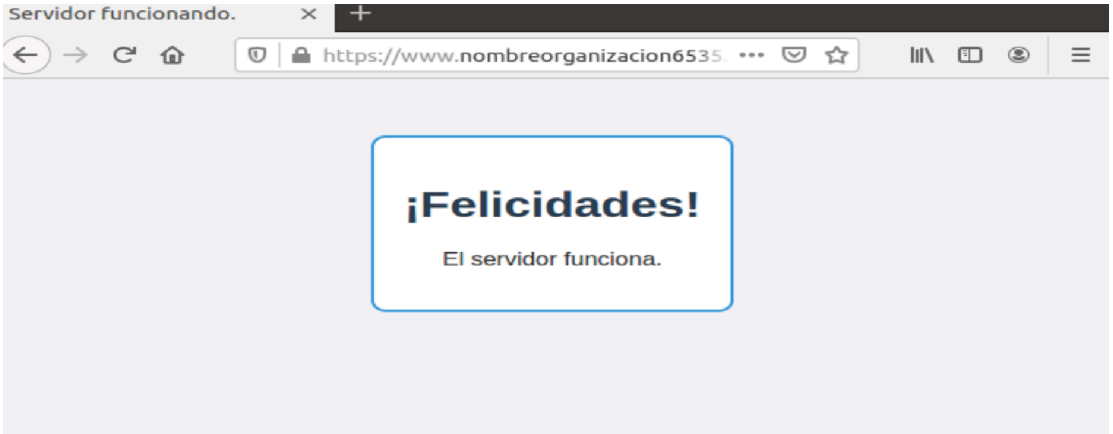


imagen 23: verificación funcionamiento HTTPS

5. IPSec

En este apartado se explicará lo implementado en el proyecto relacionado con la implementación del estándar IPSec.

Para ello, hemos modificado principalmente 2 ficheros y un directorio. Ambos en la máquina cliente y servidor.

El primero a mencionar es el fichero ipsec.conf. Este fichero contiene la configuración general de IPSec. Hemos añadido la conexión por defecto (default) y la cliente-servidor que es la que permitirá la conexión segura entre las dos máquinas virtuales mediante el uso de certificados.

```
  GNU nano 2.5.3                          Archivo: ipsec.conf

# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config setup
        # strictcrlpolicy=yes
        # uniqueids = no

# Add connections here.
conn %default
        ikelifetime=60m         #Tiempo de vida de una IKE SA
        keylife=20m             #Tiempo de vida de una Ipsec sa
        rekeymargin=3m
        keyingtries=1
        mobike=no
        keyexchange=ikev2       #Usamos IKEv2
        authby=pubkey

#Para la conexión específica entre estos dos hosts
conn cliente-servidor
        left=192.168.230.5      #left = local (el dispositivo en el que estoy editando el .conf)
        leftcert=/etc/ipsec.d/certs/servercert.pem
        leftid="C=ES, ST=Murcia, O=UMU, OU=nombreorganizacion6535, CN=www.nombreorganizacion6535.org"
        right=192.168.230.4     #right = remote (con el que va a hablar)
        rightid="C=ES, ST=Murcia, O=UMU, OU=nombreorganizacion6535, CN=gonzalo6535.org, emailAddress=glucenavazquez@gmail.com"
        type=transport          #modo transporte
        auto=start              #IKEv2 se ejecuta en el momento que haya un ipsec start
        ah=sha1
```

```
  GNU nano 2.5.3                          Archivo: ipsec.conf

# ipsec.conf - strongSwan IPsec configuration file

# basic configuration

config setup
        # strictcrlpolicy=yes
        # uniqueids = no

# Add connections here.
conn %default
        ikelifetime=60m
        keylife=20m
        rekeymargin=3m
        keyingtries=1
        mobike=no
        keyexchange=ikev2
        authby=pubkey

conn cliente-servidor
        left=192.168.230.4
        leftcert=/etc/ipsec.d/certs/clientcert.pem
        leftid="C=ES, ST=Murcia, O=UMU, OU=nombreorganizacion6535, CN=gonzalo6535.org, emailAddress=glucenavazquez@gmail.com"
        right=192.168.230.5
        rightid="C=ES, ST=Murcia, O=UMU, OU=nombreorganizacion6535, CN=www.nombreorganizacion6535.org"
        type=transport
        auto=start
        ah=sha1
```

imágenes 24 y 25: ipsec.conf del servidor y del cliente respectivamente

El segundo de los ficheros mencionados es el fichero ipsec.secrets. Su función principal es la de almacenar la ubicación de la clave privada del dispositivo de forma segura garantizando que el sistema pueda autenticarse al establecer la conexión.

```
GNU nano 2.5.3          Archivo: /etc/ipsec.secrets

# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.
: RSA /etc/ipsec.d/private/clientkey.pem
```

```
GNU nano 2.5.3          Archivo: /etc/ipsec.secrets

# This file holds shared secrets or RSA private keys for authentication.

# RSA private key for this host, authenticating it to any other host
# which knows the public part.
: RSA /etc/ipsec.d/private/serverkey.pem
```

imágenes 26 y 27: ipsec.secrets del cliente y del servidor respectivamente

El último paso es modificar el directorio ipsec.d en el cual hay a su vez varios directorios. Para configurarlo correctamente hemos incluido en la carpeta certs, cacerts y private los ficheros correspondientes según la máquina (servercert.pem o clientcert.pem, cacert.pem y serverkey.pem o clientkey.pem respectivamente.)

Por último, habiendo realizado todo lo anterior para comprobar si todo funciona correctamente, una vez hecho el “ipsec restart” correspondiente se debe utilizar el comando “ipsec statusall”. La siguiente salida nos demuestra el correcto funcionamiento de IPSec.

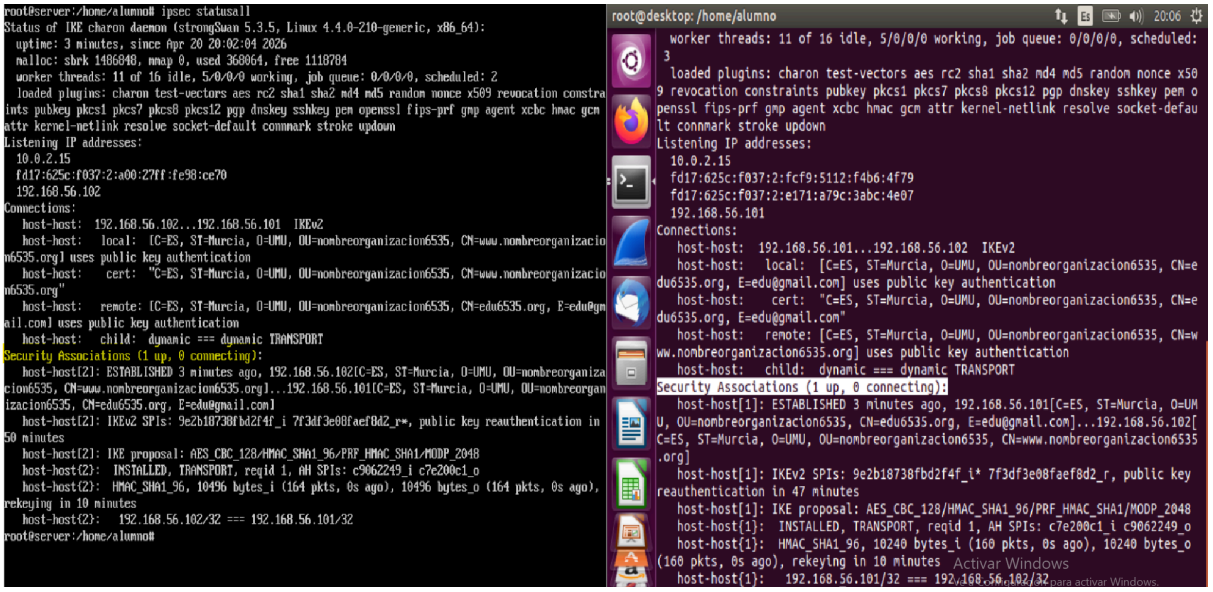


imagen 28: verificación funcionamiento IPsec

Además, se puede comprobar también que todo funciona a la perfección y que se está usando AH haciendo un simple ping del cliente al servidor y utilizando el filtro “isakmp || ah” con la herramienta wireshark verificando en su contenido si realmente usa AH. Todo esto se muestra en el apartado 7, imagen 51.

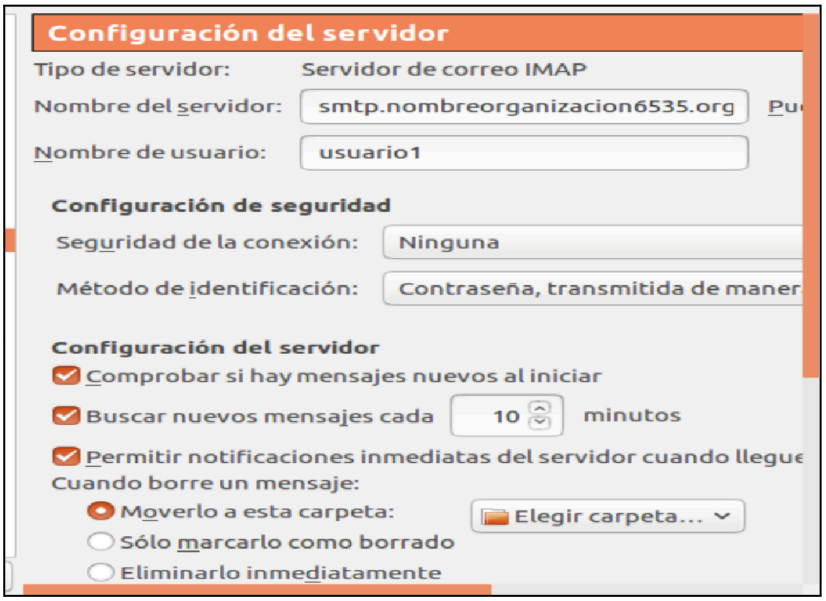
6. Funcionalidad opcional añadida

6.1. Instalar y configurar IMAP como servidor de correo entrante

Primeramente hemos instalado el módulo de IMAP de Dovecot haciendo un apt-get install dovecot-imapd y posteriormente reiniciamos el servicio para que cargue este nuevo protocolo que hemos instalado con systemctl restart dovecot.

Tras esto, pasamos a la configuración directamente sobre Thunderbird en la máquina cliente. Cogemos una de las cuentas que teníamos (por ejemplo, la de usuario1) y la eliminamos, ya que usaba el protocolo POP y ahora la vamos a crear de nuevo pero para que use IMAP.

Creamos entonces la cuenta de usuario1 como de costumbre pero, en la configuración manual debemos hacer algunos ajustes. Concretamente, en Servidor de entrada debemos seleccionar IMAP en vez de POP y como puerto el 143 (que es el que usa IMAP). El Servidor de salida lo dejamos con SMTP como lo teníamos previamente.



imágenes 29 y 30: configuración de la cuenta usuario1

Tras esto, ya tendríamos en funcionamiento completamente nuestro servidor IMAP de correo. Además, para corroborar su implementación correcta, se ofrecen unas capturas de trazas de Wireshark efectuadas tras terminar la configuración de la cuenta de usuario1 (en la sección de trazas de Wireshark).

6.2. Desplegar un dominio de resolución inversa

A diferencia de una resolución directa, con la inversa preguntamos mediante un dig dando la IP y se nos devuelve el nombre de la máquina.

Básicamente el procedimiento radica en invertir los 3 primeros octetos de nuestra dirección de red y añadiendo la terminación oficial .in-addr-arpa construir nuestra zona nueva. Es decir, si la dirección es, por ejemplo, 192.168.230.0/24, entonces la zona se llamará 230.168.192.in-addr.arpa.

Estos son los pasos a realizar para configurarlo en el servidor DNS primario:
→ En named.conf.local añadimos la nueva zona y su tipo, que será master.

```
GNU nano 2.5.3                    Archivo: named.conf.local

//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "nombreorganizacion6535.org." {
        type master;
        file "/etc/bind/db.nombreorganizacion6535.org.zone";
        allow-transfer { 192.168.230.6; };
};
zone "230.168.192.in-addr.arpa" {
        type master;
        file "/etc/bind/db.192.168.230.zone";
};
```

imagen 31: configuración de named.conf.local del DNS primario

→ A continuación creamos el archivo de zona db.192.168.230.zone y lo configuramos de la siguiente manera.

Al inicio indicamos que es un reverse data file (específico para la resolución inversa) y para la red 192.168.230.X. Dejamos el último bloque incompleto para luego solo tener que poner el último número de la IP, el 5 (de nuestro servidor DNS primario) y el 6 (de nuestro DNS secundario), y decirle a qué nombre apuntan mediante el registro PTR, que es el registro clave a usar en resolución inversa de DNS.

```
GNU nano 2.5.3                    Archivo: db.192.168.230.zone

;
; BIND reverse data file for 192.168.230.X network
;
$TTL    604800
@       IN      SOA     servidor.nombreorganizacion6535.org. admin.nombreorganizacion6535.org. (
                              1         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      servidor.nombreorganizacion6535.org.
@       IN      NS      servidor2.nombreorganizacion6535.org.

; Registros PTR (Punteros)
5       IN      PTR     servidor.nombreorganizacion6535.org.
6       IN      PTR     servidor2.nombreorganizacion6535.org.
```

imagen 32: configuración del fichero de zona db.192.168.230.zone

→ Tras eso simplemente realizamos un systemctl restart bind9 para aplicar estos cambios.

Para probarlo, en nuestra máquina cliente realizamos un dig -x 192.168.230.5 para comprobar el funcionamiento correcto de la resolución inversa.

Vemos que efectivamente en la Answer Section nuestro servidor DNS ha respondido con el registro PTR y el nombre que ofrecimos.

```
root@desktop:/home/alumno/Descargas# dig -x 192.168.230.5

; <<>> DiG 9.10.3-P4-Ubuntu <<>> -x 192.168.230.5
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 4059
;; flags: qr aa rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 2, ADDITIONAL: 3

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;5.230.168.192.in-addr.arpa.    IN      PTR

;; ANSWER SECTION:
5.230.168.192.in-addr.arpa. 604800 IN   PTR     servidor.nombreorganizacion6535.
org.

;; AUTHORITY SECTION:
230.168.192.in-addr.arpa. 604800 IN     NS      servidor.nombreorganizacion6535.
org.
230.168.192.in-addr.arpa. 604800 IN     NS      servidor2.nombreorganizacion6535
.org.

;; ADDITIONAL SECTION:
servidor.nombreorganizacion6535.org. 604800 IN A 192.168.230.5
servidor2.nombreorganizacion6535.org. 604800 IN A 192.168.230.6

;; Query time: 1 msec
;; SERVER: 192.168.230.5#53(192.168.230.5)
;; WHEN: Sat Apr 11 18:10:05 CEST 2026
```

imagen 33: comprobación de la resolución inversa

6.3. Crear un subdominio para nombreorganizacion6535.org e instalar un DNS para el mismo

Aquí, en vez de clonar de nuevo nuestra máquina servidor para establecer un nuevo servidor DNS, hacemos que nuestro servidor DNS secundario actúe como DNS primario de este nuevo subdominio. Concretamente lo vamos a llamar:
subdominio.nombreorganizacion6535.org

En primer lugar, en el fichero de zona de nuestro dominio principal en el servidor DNS primario (db.nombreorganizacion6535.org.zone) debemos establecer que se delegue el nuevo subdominio al DNS secundario, e incrementar en 1 el Serial para notificarle que hay una nueva versión del fichero de zona:

```
;
@       IN      NS      servidor.nombreorganizacion6535.org.
@       IN      NS      servidor2.nombreorganizacion6535.org.
@       IN      MX      10      smtp.nombreorganizacion6535.org.

servidor        IN      A       192.168.230.5
servidor2       IN      A       192.168.230.6

cliente IN      A       192.168.230.5
www     IN      A       192.168.230.5
web     IN      A       192.168.230.5
smtp    IN      A       192.168.230.5
pop     IN      A       192.168.230.5
dns1    IN      A       192.168.230.5
dns2    IN      A       192.168.230.5
subdominio          IN      NS      ns1.subdominio.nombreorganizacion6535.org.
ns1.subdominio      IN      A       192.168.230.6
```

imagen 34: configuración del fichero de zona para delegar el subdominio

Tras eso, como de costumbre, hacemos systemctl restart bind9 para aplicar los cambios.

Ahora, en el DNS secundario, creamos el nuevo DNS en el que este actuará como primario. Cabe mencionar que no habrá un backup de este nuevo DNS, ya que no hace falta.
En el named.conf.local añadimos el nombre de zona del subdominio y su type master:

```
  GNU nano 2.5.3                    Archivo: named.conf.local

//
// Do any local configuration here
//

// Consider adding the 1918 zones here, if they are not used in your
// organization
//include "/etc/bind/zones.rfc1918";

zone "nombreorganizacion6535.org." {
        type slave;
        file "/var/cache/bind/db.nombreorganizacion6535.org.zone";
        masters { 192.168.230.5; };
};
zone "subdominio.nombreorganizacion6535.org" {
        type master;
        file "/etc/bind/db.subdominio.zone";
};
```

imagen 35: configuración del named.conf.local del servidor DNS secundario (que aquí actúa como principal)

Tras eso, creamos el fichero de zona para el subdominio: db.subdominio.zone
Ponemos para el dominio de pc1 una dirección IP cualquiera, en nuestro caso le hemos puesto la 100:

```
  GNU nano 2.5.3                    Archivo: db.subdominio.zone

$TTL    604800
@       IN      SOA     ns1.subdominio.nombreorganizacion6535.org. admin.subdominio.nombreorganizacion6535.org. (
                              1         ; Serial
                         604800         ; Refresh
                          86400         ; Retry
                        2419200         ; Expire
                         604800 )       ; Negative Cache TTL
;
@       IN      NS      ns1.subdominio.nombreorganizacion6535.org.

ns1     IN      A       192.168.230.6
pc1     IN      A       192.168.230.100
```

imagen 36: archivo de zona de nuestro subdominio

Tras eso, hacemos aquí el systemctl restart bind9 para aplicar los cambios y, por último, iremos a nuestra máquina cliente y realizaremos un dig para nuestro subdominio: dig pc1.subdominio.nombreorganizacion6535.org. Y, efectivamente en la Answer Section se nos devuelve la dirección inventada que le habíamos puesto al subdominio con pc1:

```
root@desktop:/home/alumno/Descargas# dig pc1.subdominio.nombreorganizacion6535.o
rg

; <<>> DiG 9.10.3-P4-Ubuntu <<>> pc1.subdominio.nombreorganizacion6535.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 19274
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 1, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 4096
;; QUESTION SECTION:
;pc1.subdominio.nombreorganizacion6535.org. IN A

;; ANSWER SECTION:
pc1.subdominio.nombreorganizacion6535.org. 604800 IN A 192.168.230.100

;; AUTHORITY SECTION:
subdominio.nombreorganizacion6535.org. 604800 IN NS ns1.subdominio.nombreorganiz
acion6535.org.

;; Query time: 5 msec
;; SERVER: 192.168.230.5#53(192.168.230.5)
;; WHEN: Sat Apr 11 18:46:09 CEST 2026
;; MSG SIZE  rcvd: 104
```

imagen 37: comprobación del funcionamiento correcto de nuestro subdominio

6.4. Realizar la autenticación del cliente con certificados en HTTPS

Ahora ya no solo el cliente verifica la autenticación del servidor, sino que ahora también es el cliente el que debe autenticarse (como dimos en clase de teoría con los mensajes Certificate Request del servidor y los Certificate y Certificate Verify del cliente en el Handshake Protocol).

Primero, en la máquina virtual servidor generamos el certificado del cliente utilizando nuestra CA:

```
root@server1:/home/alumno/demoCA# openssl req -new -nodes -newkey rsa:2048 -keyout clientkey.pem -out client
csr.pem
Generating a 2048 bit RSA private key
.................+++
................+++
writing new private key to 'clientkey.pem'
-----
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [ES]:ES
State or Province Name (full name) [Murcia]:Murcia
Murcia []:Murcia
Organization Name (eg, company) [UMU]:UMU
Organizational Unit Name (eg, section) [nombreorganizacion6535]:nombreorganizacion6535
Common Name (e.g. server FQDN or YOUR name) []:gonzalo6535.org
Email Address []:glucenavazquez@gmail.com
```

imagen 38: generación de la clave privada y petición de firma para el cliente

A continuación, la CA firma el certificado dándole validez hasta dentro de 400 días:

```
root@server1:/home/alumno/demoCA# openssl ca -keyfile private/cakey.pem -in clientcsr.pem -out clientcert.pem -days 400
Using configuration from /usr/lib/ssl/openssl.cnf
Enter pass phrase for private/cakey.pem:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 3 (0x3)
        Validity
            Not Before: Apr 18 07:43:40 2026 GMT
            Not After : May 23 07:43:40 2027 GMT
        Subject:
            countryName               = ES
            stateOrProvinceName       = Murcia
            organizationName          = UMU
            organizationalUnitName    = nombreorganizacion6535
            commonName                = gonzalo6535.org
            emailAddress              = glucenavazquez@gmail.com
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            Netscape Comment:
                OpenSSL Generated Certificate
            X509v3 Subject Key Identifier:
                B5:40:05:63:A3:CE:B9:BB:AE:41:59:B0:8F:28:E6:9A:A9:DC:FB:84
            X509v3 Authority Key Identifier:
                keyid:D1:F6:D1:2F:B7:93:CA:22:0C:8E:54:1F:A4:E5:FA:48:98:37:E8:A1

Certificate is to be certified until May 23 07:43:40 2027 GMT (400 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
```

imagen 39: firma del certificado por parte de la CA

Lo siguiente es empaquetar el certificado para el navegador, ya que estos no entienden archivos .pem sueltos, esperan un archivo en formato PKCS#12 (.pfx).

Simplemente se nos pedirá introducir una contraseña de exportación y verificarla:

```
root@server1:/home/alumno/demoCA# openssl pkcs12 -export -in clientcert.pem -certfile cacert.pem -inkey clie
ntkey.pem -out clientcert.pfx
Enter Export Password:
Verifying - Enter Export Password:
```

imagen 40: empaquetado del certificado en formato PKCS#12

Tras esto, configuramos nuestro archivo .conf del servidor apache para que exija el certificado. Concretamente, dentro del bloque de VirtualHost *: 443 (el que hace referencia a HTTPS) añadimos las cláusulas SSLVerifyClient y SSLVerifyDepth al final:

```
<VirtualHost *:443>
ServerAdmin gonzalo@sstt.com
        ServerName www.nombreorganizacion6535.org
        DocumentRoot /var/www/nombreorganizacion6535
        <Directory /var/www/nombreorganizacion6535>
                AllowOverride None
                Order allow,deny
                allow from all
        </Directory>
        SSLEngine on
        SSLCertificateFile /home/alumno/demoCA/servercertSAN.pem
        SSLCertificateKeyFile /home/alumno/demoCA/serverkeySAN.pem
        SSLCACertificateFile /home/alumno/demoCA/cacert.pem
        SSLVerifyClient require
        SSLVerifyDepth 10
</VirtualHost>
```

imagen 41: nueva configuración del servidor apache para la autenticación de cliente

Una vez guardado el archivo, reiniciamos apache con systemctl restart apache2. Tras hacerlo, nos pasamos el archivo clientcert.pfx desde el servidor al cliente y a partir de ahí, pasamos a las configuraciones desde la máquina virtual cliente.

En Firefox, abrimos Preferencias → Privacidad y Seguridad, y ahí le damos a Ver Certificados. Ya en esta sección, le damos a Sus Certificados e importamos nuestro certificado para el cliente que nos hemos pasado desde el servidor. Deberá verse así:

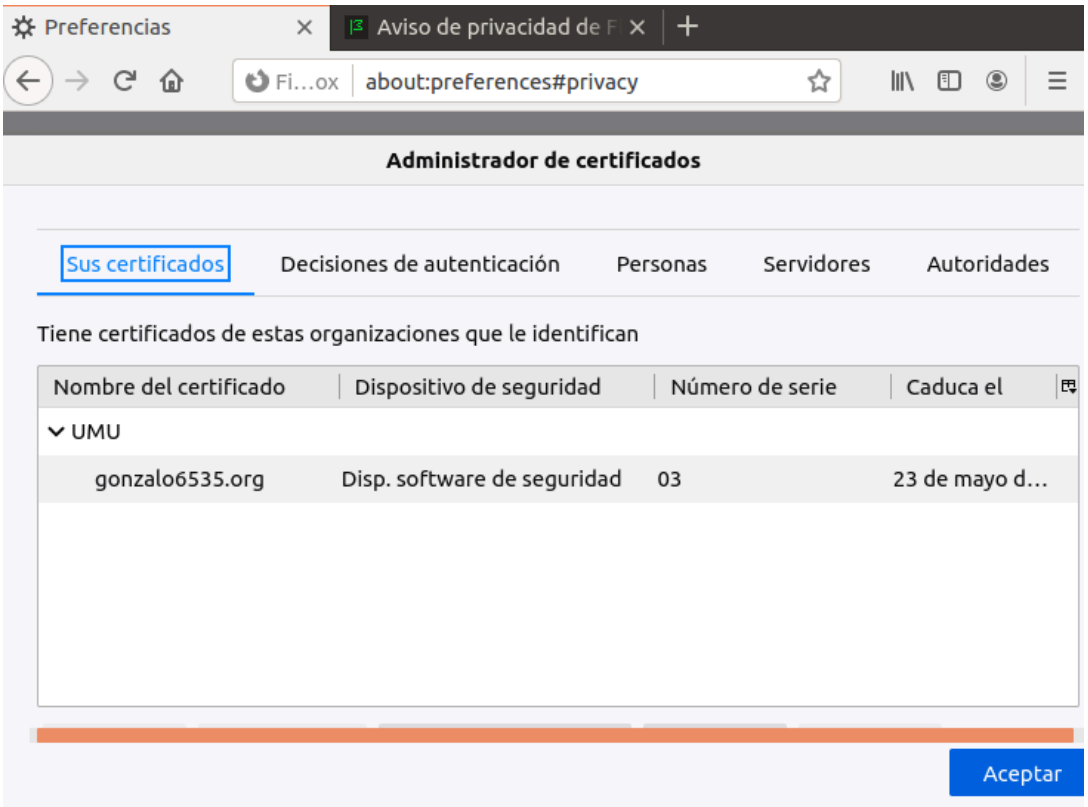


imagen 42: importación del certificado del cliente

Por último, configuramos Firefox para desactivar TLS 1.3 en el propio menú de configuración de Firefox poniendo security.tls.version.max y poniendo el valor 3.

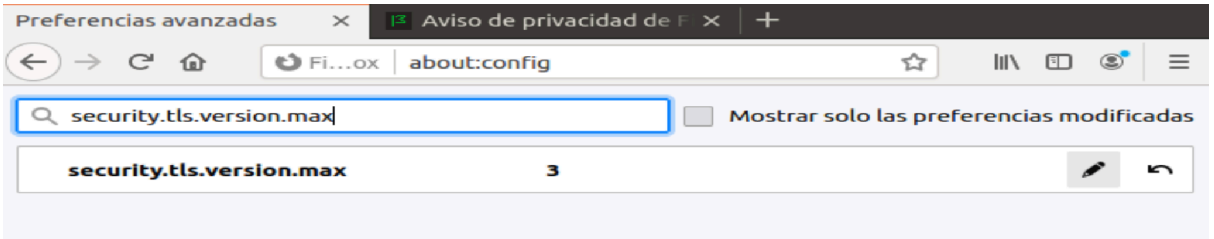


imagen 43: cambio de valor por defecto de security.tls.version.max a 3

Y ya con eso ya estaría todo configurado todo para que desde el cliente podamos acceder a https://www.nombreorganizacion6535.org pidiéndonos el servidor un certificado de autenticación. A continuación se muestran unas capturas del resultado de la búsqueda y en la sección de trazas se muestran unas trazas del intercambio de mensajes entre ambos.

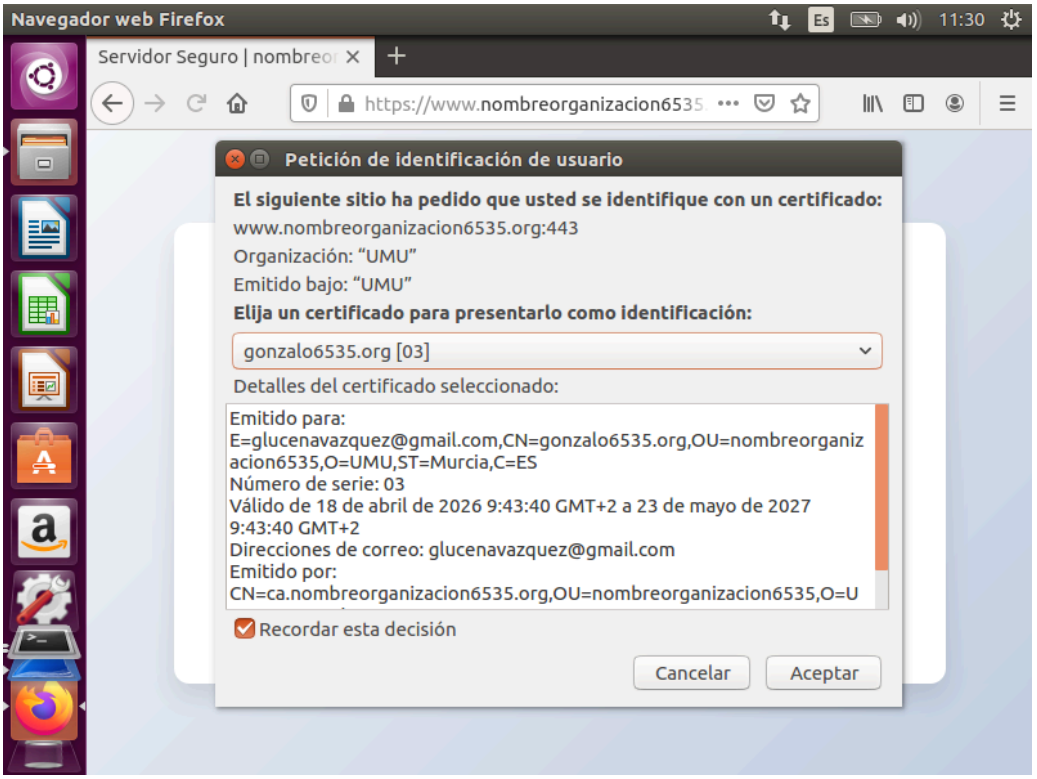


imagen 44: mensaje de solicitud del certificado por parte del servidor a través del navegador

6.5. Instalar y configurar FTP.

En primer lugar, para instalar FTP en nuestra máquina servidor ejecutamos apt install vsftpd, que es el software que utilizaremos para el uso de FTP. A continuación, abrimos el fichero de configuración vsftpd.conf para configurar los permisos. Concretamente estas son las opciones que se han tocado, el resto se han quedado de fábrica:

```
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
#
# Uncomment this to allow local users to log in.
local_enable=YES
#
# Uncomment this to enable any form of FTP write command.
write_enable=YES
#
```

imagen 45: opciones configuradas en vsftpd.conf

Tras terminar la configuración, reiniciamos el servicio con systemctl restart vsftpd.

```
tcp6       0      0 :::21                   :::*                    ESCUCHAR
```

imagen 46: al hacer netstat -na aparece el puerto 21 (el que usa FTP) en escucha

Ahora, en la máquina cliente creamos, un archivo de texto rápido (p.ej mediante un echo y redirigiéndolo a un fichero para que se cree)., el nuestro concretamente se llama prueba_ftp.txt.

Usamos el comando ftp para conectarnos con el servidor: ftp 192.168.230.5 y, tras autenticarnos como usuario alumno, el comienzo del prompt del terminal cambia a ftp>.

Hacemos put prueba_ftp.txt y nuestro archivo se transferirá a nuestro servidor:

```
ftp> put prueba_ftp.txt
local: prueba_ftp.txt remote: prueba_ftp.txt
200 PORT command successful. Consider using PASV.
150 Ok to send data.
226 Transfer complete.
25 bytes sent in 0.00 secs (79.0099 kB/s)
```

```
root@server1:/home/alumno# ls
demoCA            ficheros_server1.zip                        Maildir         SSTT-practicas
ficheros_pop      ficheros_smtp                               muttrc
ficheros_pop.zip  ficheros_smtp.zip                           prueba_ftp.txt
ficheros_server1  linux-headers-4.4.0-59_4.4.0-59.80_all.deb  sent
```

imágenes 47 y 48: transferencia del archivo de nuestro cliente hacia el servidor y comprobación con un ls

7. Trazas de Wireshark

Captura del intercambio de DNS Query y DNS Reply de la petición desde el cliente a nuestro DNS primario mediante dig www.nombreorganizacion6535.org:

1	0.000000000	192.168.230.4	192.168.230.5	DNS	101 Standard query 0x882b A www.nombreorganizacion6535.org OPT
2	0.002838703	192.168.230.5	192.168.230.4	DNS	196 Standard query response 0x882b A www.nombreorganizacion6535.org A 192.168.230.5 NS servidor.nombreorganizacion653…

imagen 49: intercambio DNS Query y DNS Reply

También aquí tenemos la captura del protocolo SMTP mediante el envío de correos usando Thunderbird:

14	19.061631308	192.168.230.4	192.168.230.5	SMTP	88 C: EHLO [192.168.230.4]
15	19.063819748	192.168.230.5	192.168.230.4	TCP	66 25 → 50768 [ACK] Seq=71 Ack=23 Win=29056 Len=0 TSval=2587837 TSecr=2528339
16	19.064666349	192.168.230.5	192.168.230.4	SMTP	186 S: 250-server1 Hello [192.168.230.4] [192.168.230.4] | SIZE 52428800 | 8BITMIME | PIPELINING | PRDR | HELP
17	19.064682363	192.168.230.4	192.168.230.5	TCP	66 50768 → 25 [ACK] Seq=23 Ack=191 Win=29312 Len=0 TSval=2528340 TSecr=2587837
18	19.125156431	192.168.230.4	192.168.230.5	SMTP	138 C: MAIL FROM:<usuario1@nombreorganizacion6535.org> BODY=8BITMIME SIZE=481
19	19.128133548	192.168.230.5	192.168.230.4	SMTP	74 S: 250 OK
20	19.128163698	192.168.230.4	192.168.230.5	TCP	66 50768 → 25 [ACK] Seq=95 Ack=199 Win=29312 Len=0 TSval=2528356 TSecr=2587853
21	19.557206055	192.168.230.4	192.168.230.5	SMTP	113 C: RCPT TO:<usuario2@nombreorganizacion6535.org>
22	19.560191624	192.168.230.5	192.168.230.4	SMTP	80 S: 250 Accepted
23	19.560222585	192.168.230.4	192.168.230.5	TCP	66 50768 → 25 [ACK] Seq=142 Ack=213 Win=29312 Len=0 TSval=2528464 TSecr=2587961
24	19.567147029	192.168.230.4	192.168.230.5	SMTP	72 C: DATA
25	19.568908864	192.168.230.5	192.168.230.4	SMTP	122 S: 354 Enter message, ending with "." on a line by itself

25	19.568908864	192.168.230.5	192.168.230.4	SMTP	122 S: 354 Enter message, ending with "." on a line by itself
26	19.586741101	192.168.230.4	192.168.230.5	SMTP	547 C: DATA fragment, 481 bytes
27	19.588003165	192.168.230.4	192.168.230.5	SMTP/I…	69 from: usuario1 <usuario1@nombreorganizacion6535.org>, subject: Prueba V2, (text/plain)
28	19.591818240	192.168.230.5	192.168.230.4	TCP	66 25 → 50768 [ACK] Seq=269 Ack=632 Win=30080 Len=0 TSval=2587968 TSecr=2528470
29	19.623545966	192.168.230.5	192.168.230.4	SMTP	94 S: 250 OK id=1w0lzk-00012v-28
30	19.627877464	192.168.230.4	192.168.230.5	SMTP	72 C: QUIT

imágenes 50 y 51: trazas del protocolo SMTP

Captura de dig MX nombreorganizacion6535.org:

No.	Time	Source	Destination	Protocol	Lengtl	Info
9	62.149670393	192.168.56.101	192.168.56.102	DNS	97	Standard query 0x6a6c MX nombreorganizacion6535.org OPT
10	62.150791195	192.168.56.102	192.168.56.101	DNS	219	Standard query response 0x6a6c MX nombreorganizacion6535.org MX 10 smtp.nombreorganizacion6535.org NS servidor.nombreorganizacion6535…

imagen 52: captura de dig MX nombreorganizacion6535.org

Se puede ver como el cliente contacta con el servidor y este le responde de manera simple.

Captura de dig www.um.es:

No.	Time	Source	Destination	Protocol	Lengtl	Info
1	0.000000	192.168.56.101	192.168.56.102	DNS	82	Standard query 0xdc42 A www.um.es OPT
2	0.000582	10.0.2.15	192.203.230.10	DNS	82	Standard query 0xa495 A www.um.es OPT
4	0.010501	192.203.230.10	10.0.2.15	DNS	549	Standard query response 0xa495 A www.um.es NS a.nic.es NS c.nic.es NS g.nic.es NS h.nic.es DS RRSIG A 194.69.254.1 A 194.0.34.53 A 20…
6	0.010750	10.0.2.15	194.0.33.53	DNS	82	Standard query 0xde8d A www.um.es OPT
18	0.076466	194.0.33.53	10.0.2.15	DNS	82	Standard query response 0xde8d A www.um.es OPT
22	0.142783	10.0.2.15	194.0.33.53	DNS	96	Standard query 0x043d A www.um.es OPT
24	0.208834	194.0.33.53	10.0.2.15	DNS	1115	Standard query response 0x043d A www.um.es NS sun.rediris.es NS chico.rediris.es NS dns2.um.es NS dns1.um.es NSEC3 RRSIG NSEC3 RRSIG …
26	0.210009	fd17:625c:f037:2:a0…	2001:720:1710:1::2	DNS	102	Standard query 0xb995 A www.um.es OPT
28	0.210873	fe80::2	fd17:625c:f037:2:a0…	ICMPv6	150	Destination Unreachable (No route to destination)
33	1.010150	10.0.2.15	155.54.1.2	DNS	82	Standard query 0x1236 A www.um.es OPT
34	1.037411	155.54.1.2	10.0.2.15	DNS	119	Standard query response 0x1236 A www.um.es CNAME wwwclu.um.es A 155.54.212.103 OPT
37	1.081942	192.168.56.102	192.168.56.101	DNS	379	Standard query response 0xdc42 A www.um.es CNAME wwwclu.um.es A 155.54.212.103 NS sun.rediris.es NS chico.rediris.es NS dns2.um.es NS…

imagen 53: captura de dig www.um.es

En esta captura se puede observar como va pidiendo de manera iterativa los recursos que necesita empezando por .es, después um.es y finalmente www.um.es.

Captura de recibo de correo:

No.	Time	Source	Destination	Protocol	Lengtl	Info
8	0.012031728	192.168.56.102	192.168.56.101	POP	86	S: +OK Dovecot ready.
10	0.012393094	192.168.56.101	192.168.56.102	POP	72	C: CAPA
12	0.013614513	192.168.56.102	192.168.56.101	POP	149	S: +OK
13	0.014006699	192.168.56.101	192.168.56.102	POP	78	C: AUTH PLAIN
14	0.015755647	192.168.56.102	192.168.56.101	POP/IMF	70	+
15	0.015955972	192.168.56.101	192.168.56.102	POP	92	C: AHVzdWFyaW8yAHVzdWFyaW8y
16	0.030511584	192.168.56.102	192.168.56.101	POP	82	S: +OK Logged in.
17	0.030713877	192.168.56.101	192.168.56.102	POP	72	C: STAT
18	0.040693979	192.168.56.102	192.168.56.101	POP	77	S: +OK 1 844
19	0.042230334	192.168.56.101	192.168.56.102	POP	72	C: LIST
20	0.044053202	192.168.56.102	192.168.56.101	POP	93	S: +OK 1 messages:
21	0.045081078	192.168.56.101	192.168.56.102	POP	72	C: UIDL
22	0.046384958	192.168.56.102	192.168.56.101	POP	94	S: +OK
23	0.046898692	192.168.56.101	192.168.56.102	POP	74	C: RETR 1
24	0.048182359	192.168.56.102	192.168.56.101	POP	929	S: +OK 844 octets
25	0.069944262	192.168.56.101	192.168.56.102	POP	72	C: QUIT
26	0.076525947	192.168.56.102	192.168.56.101	POP	84	S: +OK Logging out.

imagen 54: captura de recibo de correo

Aquí se ve como se pide el recurso y se envía correctamente.

Trazas del intercambio de mensajes tras haber implementado IPSec y hacer un ping a nuestro servidor (imagen 51) y al acceder a la página web con el servicio Apache activo (imagen 52).

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

isakmp || ah Expression...

No.	Time	Source	Destination	Protocol	Length	Info
2	7.425495208	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
3	7.426405354	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
4	8.427316489	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
5	8.428486315	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
6	9.429883885	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
7	9.430737513	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
8	10.436583494	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
9	10.438022648	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
10	11.443268538	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
11	11.444945065	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
14	12.446981025	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
15	12.448535512	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
16	13.448453010	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
17	13.450652896	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
18	14.450517682	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
19	14.451307009	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
20	15.452330233	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
21	15.453255623	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
22	16.458942051	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
23	16.462837405	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply
24	17.461070270	192.168.230.4	192.168.230.5	ICMP	122	Echo (ping) request
25	17.462293374	192.168.230.5	192.168.230.4	ICMP	122	Echo (ping) reply

```
Reserved: 0000
AH SPI: 0xc9e00be9
AH Sequence: 35
AH ICV: ddd074c7ce2a01918e1e143e

0000  08 00 27 da a9 89 08 00  27 88 18 17 08 00 45 00   ··'····· '·····E·
0010  00 6c 93 f0 40 00 40 33  59 13 c0 a8 e6 04 c0 a8   ·l··@·@3 Y······ 
```

Ready to load or capture Packets: 25 · Displayed: 22 (88.0%) Profile: Default

imagen 55: trazas de verificación del funcionamiento de IPSec con AH

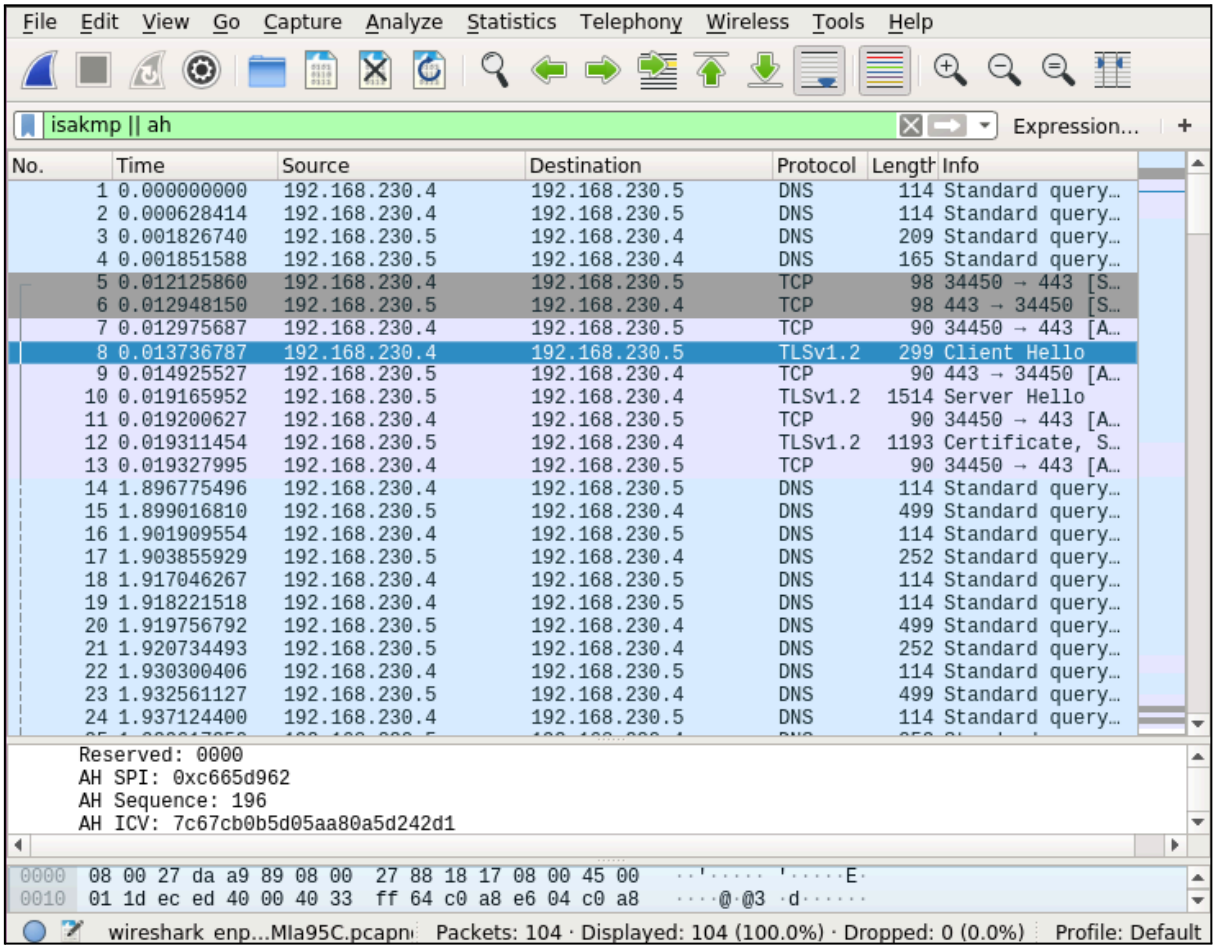
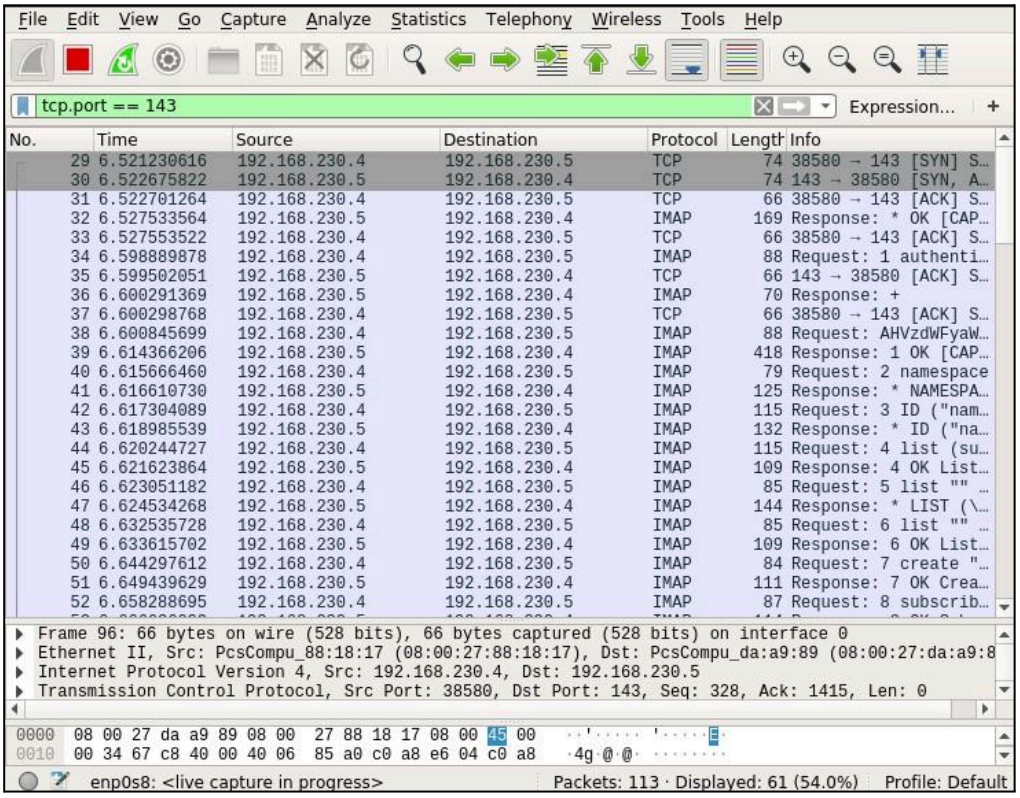
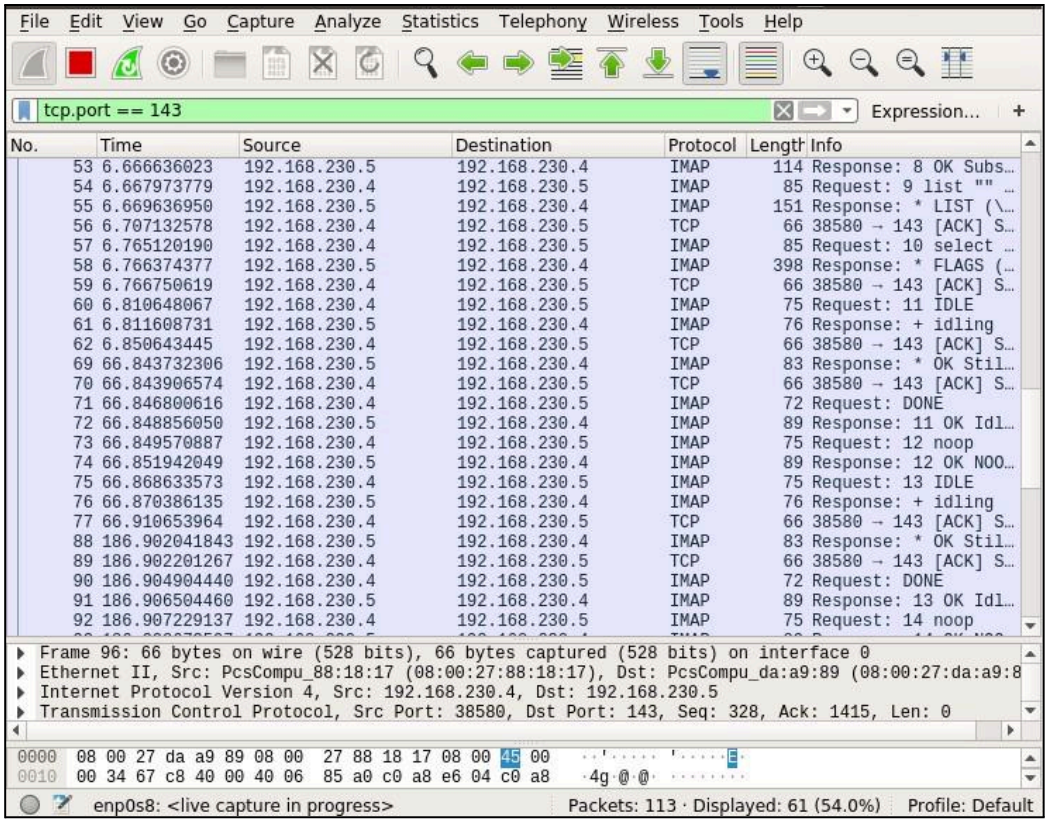


imagen 56: visualización de todos los protocolos funcionando conjuntamente

Captura de los intercambios entre el servidor de correo IMAP y el cliente:





imágenes 57 y 58: intercambios entre el servidor de correo IMAP y el cliente

Aquí podemos ver los el intercambio de mensajes del Handshake Protocol entre el servidor y el cliente a la hora del cliente haciendo una consulta https a www.nombreorganizacion.org, centrándonos especialmente en los mensajes de autenticación del cliente (mensaje Certificate Request por parte del servidor al cliente y el Certificate Verify del cliente al servidor)

tcp.port == 443 | Expression...

No.	Time	Source	Destination	Protocol	Length	Info
31	11.933496308	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [ACK] S…
32	11.933903841	192.168.230.4	192.168.230.5	TLSv1.2	279	Client Hello
33	11.934482070	192.168.230.5	192.168.230.4	TCP	66	443 → 42760 [ACK] S…
34	11.937408730	192.168.230.5	192.168.230.4	TLSv1.2	2565	Server Hello, Certi…
35	11.937424697	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [ACK] S…
40	21.954007921	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
41	21.954977950	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
46	31.969704727	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
47	31.970863154	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
48	41.986481551	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
49	41.987474855	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
50	52.001608148	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
51	52.002651595	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
54	62.017834747	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
55	62.018921981	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
58	72.034245772	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
59	72.035229072	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
60	74.774831227	192.168.230.4	192.168.230.5	TLSv1.2	1490	Certificate, Client…
61	74.777310370	192.168.230.5	192.168.230.4	TLSv1.2	1380	New Session Ticket,…
62	74.777329073	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [ACK] S…
63	74.796327197	192.168.230.4	192.168.230.5	TLSv1.2	97	Encrypted Alert
64	74.797442460	192.168.230.5	192.168.230.4	TLSv1.2	97	Encrypted Alert
65	74.797448586	192.168.230.5	192.168.230.4	TCP	66	443 → 42760 [FIN, A…
66	74.798426794	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [RST, A…

```
Length: 167
▸ Handshake Protocol: Certificate Request
▸ Handshake Protocol: Server Hello Done
```

tcp.port == 443 | Expression... +

No.	Time	Source	Destination	Protocol	Length	Info
31	11.933496308	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [ACK] S…
32	11.933903841	192.168.230.4	192.168.230.5	TLSv1.2	279	Client Hello
33	11.934482070	192.168.230.5	192.168.230.4	TCP	66	443 → 42760 [ACK] S…
34	11.937408730	192.168.230.5	192.168.230.4	TLSv1.2	2565	Server Hello, Certi…
35	11.937424697	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [ACK] S…
40	21.954007921	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
41	21.954977950	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
46	31.969704727	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
47	31.970863154	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
48	41.986481551	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
49	41.987474855	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
50	52.001608148	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
51	52.002651595	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
54	62.017834747	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
55	62.018921981	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
58	72.034245772	192.168.230.4	192.168.230.5	TCP	66	[TCP Keep-Alive] 42…
59	72.035229072	192.168.230.5	192.168.230.4	TCP	66	[TCP Keep-Alive ACK…
60	74.774831227	192.168.230.4	192.168.230.5	TLSv1.2	1490	Certificate, Client…
61	74.777310370	192.168.230.5	192.168.230.4	TLSv1.2	1380	New Session Ticket,…
62	74.777329073	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [ACK] S…
63	74.796327197	192.168.230.4	192.168.230.5	TLSv1.2	97	Encrypted Alert
64	74.797442460	192.168.230.5	192.168.230.4	TLSv1.2	97	Encrypted Alert
65	74.797448586	192.168.230.5	192.168.230.4	TCP	66	443 → 42760 [FIN, A…
66	74.798426794	192.168.230.4	192.168.230.5	TCP	66	42760 → 443 [RST, A…

```
Length: 1368
▸ Handshake Protocol: Certificate
▸ Handshake Protocol: Client Key Exchange
▸ Handshake Protocol: Certificate Verify

0000  08 00 27 da a9 89 08 00  27 88 18 17 08 00 45 00   ··'····· '·····E·
0010  05 c4 30 71 40 00 40 06  b7 67 c0 a8 e6 04 c0 a8   ··0q@·@· ·g******
```

enp0s8: <live capture in progress> | Packets: 105 · Displayed: 34 (32.4%) | Profile: Default

imágenes 59 y 60: intercambio de mensajes del Handshake Protocol entre el servidor y el cliente

Traza del funcionamiento de FTP (además se puede apreciar cómo aparece el Autentication Header, que demuestra que el paquete FTP está encapsulado y protegido por IPSec):

ip.addr == 192.168.230.4 and ip.addr == 192.168.230.5 Expression...

No.	Time	Source	Destination	Protocol	Length	Info
50	3.145798990	192.168.230.5	192.168.230.4	FTP	113	Response: 230 …
51	3.146060374	192.168.230.4	192.168.230.5	TCP	90	39466 → 21 [AC…
54	3.147445549	192.168.230.4	192.168.230.5	FTP	96	Request: SYST
56	3.148282545	192.168.230.5	192.168.230.4	FTP	109	Response: 215 …
62	3.186571362	192.168.230.4	192.168.230.5	TCP	90	39466 → 21 [AC…
123	15.950500477	192.168.230.4	192.168.230.5	FTP	98	Request: TYPE I
125	15.951377643	192.168.230.5	192.168.230.4	FTP	121	Response: 200 …
127	15.951941918	192.168.230.4	192.168.230.5	TCP	90	39466 → 21 [AC…
128	15.952331669	192.168.230.4	192.168.230.5	FTP	118	Request: PORT …
129	15.953299318	192.168.230.5	192.168.230.4	FTP	141	Response: 200 …
132	15.954724912	192.168.230.4	192.168.230.5	FTP	112	Request: STOR …
135	15.955775370	192.168.230.5	192.168.230.4	TCP	98	20 → 50681 [SY…
136	15.955799784	192.168.230.4	192.168.230.5	TCP	98	50681 → 20 [SY…
137	15.956233700	192.168.230.5	192.168.230.4	TCP	90	20 → 50681 [AC…
138	15.956919063	192.168.230.5	192.168.230.4	FTP	112	Response: 150 …
139	15.957223967	192.168.230.4	192.168.230.5	FTP-DA…	117	FTP Data: 27 b…

```
▶ Internet Protocol Version 4, Src: 192.168.230.4, Dst: 192.168.230.5
▶ Authentication Header
▶ Transmission Control Protocol, Src Port: 50681, Dst Port: 20, Seq: 1, Ack: 1, Len: 27
  FTP Data (27 bytes data)
  [Setup frame: 128]
  [Setup method: PORT]
  [Command: STOR prueba2_ftp.txt]
  Command frame: 132
  [Current working directory: ]
▼ Line-based text data (1 lines)
    Prueba de subida 2 por FTP\n
```

imagen 61: intercambio FTP y muestra del mensaje FTP Data con el contenido del archivo

8. Problemas encontrados

Ha habido varias piedras sobre las que nos hemos tropezado en la realización de la práctica, aunque nada que no haya sido solventable preguntando al profesor o buscando en internet. Algunos de ellos son:

→ Problemas con las cuentas de correo en ThunderBird, ya que si no se ejecutaba correctamente el comando useradd con la opción especificada, las cuentas no se crean correctamente. Además de que al inicio nos costó darnos cuenta de que para recibir los correos hay que darle a un botón, no se reciben automáticamente.
→ En la parte de HTTPS nos hemos encontrado diversos problemas, especialmente relacionados con los certificados digitales y su importación a Firefox. Y es que, por ejemplo, para la mejora opcional de autenticación del cliente mediante certificados, al importar el certificado .pfx se tenía que importar obligatoriamente el certificado .pem de la CA (que no estaba, aunque se importó en sesiones de prácticas anteriores). Ese es un detalle en el que se perdió mucho tiempo.

→ En IPSec sucedió algo similar, al hacer el famoso ipsec statusall y que saliese el famoso "0 up, 0 connecting", que se podía deber por una ligera errata en el ipsec.secrets o en el ipsec.conf y que no se pudiese comprobar de otra manera que no fuese ir mirando poco a poco los ficheros. Y de la mejora opcional de autenticación de cliente derivó otro problema, pero este con IPSec: y es que el certificado clientcert.pfx no se podía usar ya que Strongswan no podía leer archivos .pfx y al final tuvimos que resignarnos y usar el clientcert.pem.

9. Conclusión y valoración personal

Conclusión: Esta práctica ha sido completamente interesante debido al factor realista que esta contiene. Aunque el proyecto se trate de una infraestructura muy minimizada los protocolos y herramientas utilizadas son las de la vida real. El ver cómo íbamos implementando "pieza a pieza" el escenario completo ha resultado muy gratificante al tener todo completo.

La práctica en sí ayuda enormemente a comprender los contenidos teóricos de la asignatura los cuales se explican paralelamente cada semana lo cual es un acierto.

Para terminar, los apartados extra realizados son de gran interés permitiendo aprender un poco más de este campo a los alumnos más interesados en la asignatura.

Valoración personal: A nivel personal, esta práctica es un acierto. Nuestra experiencia es de las mejores en la universidad ya que no contiene cargas de trabajo excesivas ni conceptos que no tienen nada que ver con lo visto en teoría. Además, las sesiones de los laboratorios son de gran utilidad ya que se explica paso a paso cómo realizar cada apartado de la práctica y, prácticamente, el 90% del trabajo se realiza en las sesiones de prácticas.
Estamos muy satisfechos con este proyecto y os animamos a seguir así.

Horas de trabajo:

- Sesiones de laboratorio: 20 horas (12 sesiones de 1 hora y 40 minutos).
- Trabajo autónomo: 26 horas (unas 2 horas a la semana durante 13 semanas).
- Verificaciones: 3 horas (1 hora por verificación).
- Total: 49 horas aproximadamente.

10. Bibliografía

Internet Systems Consortium. (s.f.). *BIND 9 Administrator Reference Manual*. Recuperado el 22 de abril de 2026, de https://bind9.readthedocs.io/

Pereñíguez García, F. (2026). *Apuntes de las sesiones teóricas y prácticas de Servicios Telemáticos* [Notas de clase no publicadas]. Departamento de Ingeniería de la Información y las Comunicaciones, Universidad de Murcia.

Python Software Foundation. (s.f.). *selectors — High-level I/O multiplexing*. Python 3 Documentation. Recuperado el 22 de abril de 2026, de https://docs.python.org/3/library/selectors.html

The Apache Software Foundation. (s.f.). *Apache HTTP Server Version 2.4 Documentation*. Recuperado el 22 de abril de 2026, de https://httpd.apache.org/docs/2.4/

The strongSwan Project. (s.f.). *strongSwan Documentation* (Configuración de IPSec). Recuperado el 22 de abril de 2026, de https://docs.strongswan.org/